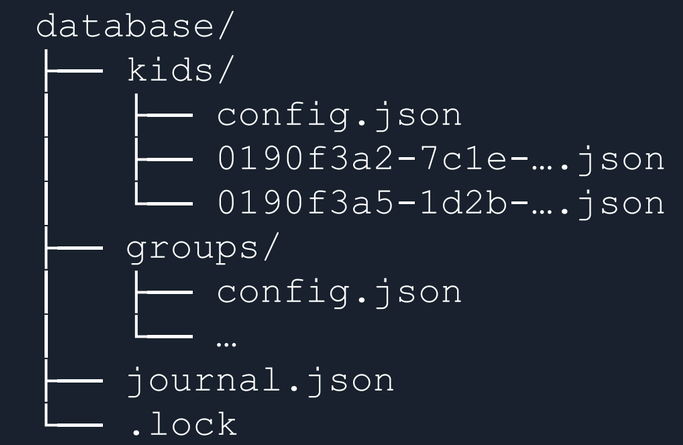
Файлова база даних

Власний рушій на JSON-файлах для Kindergarten API: без SQL, без NoSQL, без готових файлових баз



Челендж: дані живуть лише в локальних файлах

> *При розробці серверної сторони заборонено використовувати будь-яку готову базу даних (SQL, NoSQL). Всі дані повинні жити в локальних файлах (наприклад, json).*
>
> — Архітектурне обмеження від Івана

~5 300

рядків коду рушія разом із JSDoc

20

модулів у src/storage

~720

тест-кейсів у проєкті, 28 файлів — лише рушій

2

зовнішні пакети в рушії: uuid і http-status-codes

План

01 Файлова структура

02 config.json

03 Модулі рушія

04 Індекси

05 Рушій запитів

06 Зв’язки

07 Транзакції

08 Надійність

09 Помилки й логування

10 Новий челендж

Спершу розглядали: одна таблиця = один JSON-файл

```
// database/kids.json
[
  { "id": 1, "firstName": "Оля", "age": 3 },
  { "id": 2, "firstName": "Петро", "age": 4 },
  { "id": 3, "firstName": "Ганна", "age": 5 },
  // … ще тисячі записів
]
```

Щоб змінити вік Олі, треба переписати весь файл.

- **Запис: O(N).** Зміна одного поля = прочитати, розпарсити й переписати всю таблицю
- **Пошук: O(N).** Навіть get за id парсить увесь файл
- **Конкуренція.** Будь-який запис блокує всю таблицю
- **Збій.** Обірваний запис пошкоджує всю таблицю, а не один рядок
- **Пам’ять.** Великий файл повністю потрапляє в RAM на кожну операцію

Обрано: колекція = директорія, запис = файл

```
database/
├── kids/
│   ├── config.json        ← схема, у Git
│   ├── 0190f3a2-….json    ← запис
│   └── 0190f3a5-….json
├── groups/
│   ├── config.json
│   └── …
├── journal.json           ← журнал
└── .lock                  ← блокування
```

Операція	Таблиця = файл	Запис = файл
get за id	парсити весь файл, O(N)	прочитати 1 файл, O(1)
змінити запис	переписати весь файл	переписати 1 файл
пошук за полем	парсити весь файл	індекс у RAM → лише збіги
збій посеред запису	втрачена вся таблиця	атомарний rename: 0 втрат
записи в Git	один великий diff	схеми в Git, дані — ні

Ім’я колекції = ім’я директорії. Колекція з’являється, щойно в ній є config.json.

Первинний ключ — UUIDv7, і його генерує рушій

SQL

```sql
CREATE TABLE kids (
  id BIGSERIAL PRIMARY KEY,
  ...
);

INSERT INTO kids (first_name, age)
VALUES ('Оля', 3)
RETURNING id;           -- 42

SELECT * FROM kids WHERE id = 42;
```

У НАС

```ts
// id передати не можна: його генерує рушій
const kid = await db.create('kids', {
  firstName: 'Оля', age: 3,
});
// kid.id = '0190f3a2-7c1e-7b4d-…'

await db.get('kids', kid.id);
// → readFile('database/kids/0190f3a2-….json')
```

Час у ключі

перші 48 біт UUIDv7 — мітка часу, тож сортування за id = порядок створення

Без лічильника

id унікальний без спільного стану: не треба блокувати базу на INSERT

RAM: лише id

у пам’яті тримаються id та індекси; записи читаються з диска на вимогу

config.json — схема колекції

```
{
  "table": "groups",
  "primaryKey": "id",
  "timestamps": true,
  "columns": {
    "name": { "type": "string", "required": true,
              "unique": true, "maxLength": 100 },
    "age":  { "type": "integer", "required": true,
              "min": 2, "max": 5 },
    "educatorId": { "type": "string",
      "nullable": true, "default": null,
      "unique": true,
      "references": { "collection": "educators",
                      "onDelete": "setNull" } }
  },
  "indexes": ["age"]
}
```

`table`	збігається з іменем директорії
`primaryKey`	завжди "id"; оголосити колонку id не можна
`timestamps`	рушій сам веде createdAt і updatedAt
`columns`	колонки: тип і правила кожної
`indexes`	поля для індексу в пам’яті; може бути []

Перевірка під час старту

Невідомі ключі ("requred"), опція не до свого типу, min > max, неіснуюча колекція в references → INVALID_CONFIG, і сервер не стартує.

Типи й опції колонок і їхні аналоги в SQL

Типи

`string`	`number`
`integer`	`boolean`
`object`	`array`
`date`	`datetime`

date — YYYY-MM-DD, без поясу.
datetime — завжди в UTC, тож рядки порівнюються як моменти часу.

Опція	Що робить	Аналог у SQL
`required`	поле обов’язкове	`NOT NULL (+ немає DEFAULT)`
`nullable`	дозволено null	`NULL`
`default`	значення, якщо не передали	`DEFAULT …`
`min / max`	межі для чисел і дат	`CHECK (x BETWEEN …)`
`minLength / maxLength`	довжина рядка	`VARCHAR(n), CHECK length`
`enum`	перелік значень	`CHECK (x IN (…)) / ENUM`
`pattern`	регулярний вираз	`CHECK (x ~ '…')`
`unique`	значення не повторюється	`UNIQUE`
`references`	посилання на запис	`REFERENCES … ON DELETE`
`timestamps`	createdAt / updatedAt	`DEFAULT now() + тригер`

Та сама таблиця: CREATE TABLE vs config.json

SQL

```
CREATE TABLE kids (
  id          UUID PRIMARY KEY,
  first_name  VARCHAR(100) NOT NULL
              CHECK (length(first_name) >= 1),
  age         INTEGER NOT NULL
              CHECK (age BETWEEN 2 AND 5),
  sex         TEXT NOT NULL
              CHECK (sex IN ('MALE', 'FEMALE')),
  group_id    UUID NULL DEFAULT NULL
              REFERENCES groups (id)
              ON DELETE SET NULL,
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ
);
CREATE INDEX ON kids (age);
```

У НАС

```
{
  "table": "kids", "primaryKey": "id",
  "timestamps": true,
  "columns": {
    "firstName": { "type": "string", "required": true,
                   "minLength": 1, "maxLength": 100 },
    "age": { "type": "integer", "required": true,
             "min": 2, "max": 5 },
    "sex": { "type": "string", "required": true,
             "enum": ["MALE", "FEMALE"] },
    "groupId": { "type": "string", "nullable": true,
      "default": null,
      "references": { "collection": "groups",
                      "onDelete": "setNull" } }
  },
  "indexes": ["age"]
}
```

Міграцій немає й не буде

SQL

```
-- migrations/0007_add_speech_lessons.sql
ALTER TABLE kids
  ADD COLUMN needs_speech_lessons
  BOOLEAN NOT NULL DEFAULT false;

-- СУБД перепише всі рядки,
-- інструмент міграцій запам'ятає версію
```

У НАС

```
// 1. додати колонку в kids/config.json
"needsSpeechLessons": {
  "type": "boolean", "required": true,
  "default": false }

// 2. перезапустити сервер
// старі записи → попередження INVALID_RECORD
// 3. PATCH виправляє запис під нову схему
```

Немає DDL

Схему не змінюють командами: рушій лише читає config.json під час старту.

Старі записи працюють

Запис, що не відповідає схемі, завантажується з попередженням, а не ламає старт.

Виправлення поступові

PATCH відкидає зниклі поля й вимагає нові required-поля.

Модулі рушія та залежності між ними

file-database.js — фасад: init / close · create / get / list / update / delete · transaction

Надійність

```
process-lock.js
journal.js
fs-utils.js
```

атомарні файли, WAL, .lock

Транзакції

```
transaction-manager.js
transaction.js
lock-manager.js
```

→ Колекції, Запити

Колекції

```
collection.js
index-manager.js
validator.js
dates.js
```

→ Запити, Зв’язки, fs-utils

Запити

```
query-context.js
predicate.js
where.js
sort.js
range.js
collation.js
```

предикати, сортування, пагінація

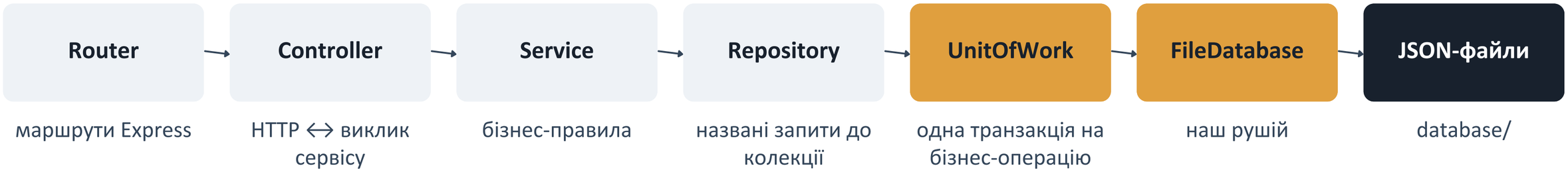
Зв’язки

```
relation-manager.js
```

граф references, onDelete

Спільне для всіх: errors.js — каталог кодів помилок і статусів · logger-contract.js — інтерфейс логера

Місце бази в застосунку



SQL

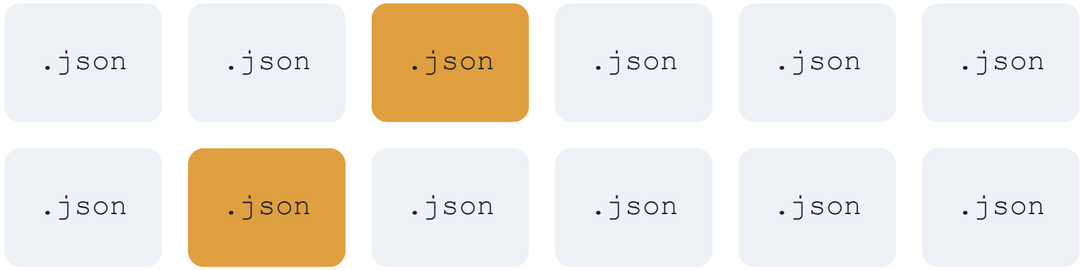
```
// типовий стек із СУБД
class KidRepository {
  findByGroup(groupId) {
    return pool.query(
      'SELECT * FROM kids WHERE group_id = $1',
      [groupId]);
  }
}
```

У НАС

```
// src/repositories/kid-repository.js
class KidRepository extends BaseRepository {
  async countByGroup(groupId) {
    const page = await this.findPage(
      { where: { groupId }, limit: 1 });
    return page.total;
  }
}
```

Індекси: навіщо вони рушію

Без індексу



readdir + 12 читань файлів, щоб знайти 2 записи

З індексом

```
// IndexManager: Map<поле, Map<значення, Set<id>>>
age → { 3 → {0190…a1, 0190…c7},
        4 → {0190…b2},
        5 → {0190…d9} }
```

Одне звернення до Map, потім 2 читання файлів

SQL

```
CREATE INDEX kids_age_idx
  ON kids (age);
```

У НАС

```
// kids/config.json
"indexes": ["age", "needsSpeechLessons"]
```

- **Лише швидкість.** Результат запиту з індексом і без нього однаковий.
- **У пам’яті.** Будуються під час старту з файлів, оновлюються на кожному коміті.
- **Масиви.** Колонка-масив індексується по елементах і прискорює contains.

Ключі й unique-поля індексуються автоматично

`references`

Посилання (groupId, motherId…)

Під час видалення групи треба знайти всі записи, що на неї посилаються (restrict / setNull). Без індексу це читання всієї колекції kids. Ці ж поля — найчастіші фільтри: where: { groupId }.

`unique`

Унікальні поля (name…)

Кожен create і update перевіряє, чи значення вже зайняте. Індекс відповідає одразу, без сканування колекції.

`items.references`

Масиви посилань (subjectIds)

Індекс по елементах: «предмет → кімнати, де він є». Потрібен для onDelete: remove і для contains.

SQL

```
group_id UUID REFERENCES groups (id),
name     TEXT UNIQUE,     -- індекс є
-- для FK індекс треба створити самому:
CREATE INDEX ON kids (group_id);
```

У НАС

```
"groupId": { "type": "string",
  "references": { "collection": "groups" } },
"name": { "type": "string", "unique": true }
// обидва поля проіндексовано автоматично
```

Фільтри: where для HTTP, предикати для коду

SQL WHERE	where (JSON, ?where=)	predicate (код сервісу)
`age = 3`	`{ age: 3 }`	`eq('age', 3)`
`group_id IS NULL`	`{ groupId: null }`	`eq('groupId', null)`
`age >= 3 AND age < 5`	`{ age: { gte: 3, lt: 5 } }`	`range('age', { gte: 3, lt: 5 })`
`role IN ('a', 'b')`	`{ role: { in: ['a', 'b'] } }`	`inList('role', ['a', 'b'])`
`$1 = ANY(child_ids)`	`{ childIds: { contains: id } }`	`contains('childIds', id)`
`a = 1 AND b <> 2`	`{ a: 1, b: { ne: 2 } }`	`and(eq('a', 1), ne('b', 2))`
`age < 3 OR speech`	—	`or(lt('age', 3), eq('speech', true))`
`довільна умова`	—	`custom((kid) => …)`

OR, вкладені умови й custom доступні лише в коді. custom завжди читає записи з диска, бо індекс йому не допоможе.

QueryContext: як виконується list()

1 Перевірка
limit 1–500,
offset ≥ 0

→

2 prepare()
поля й значення
за схемою

→

3 Фаза індексів
звузити кандидатів
у RAM

→

4 Фаза диску
перевірити решту
умов на файлах

→

5 Сортування
Intl.Collator('uk'),
кеш записів

→

6 Сторінка
slice(offset,
offset + limit)

```
// AndPredicate.evaluate: спершу індекси, потім диск
candidates = this.applyIndex(context, candidates);
if (candidates.size === 0) return candidates;
for (const id of candidates) {
  const record = await context.getRecord(id);
  if (rest.every((p) => p.test(record))) matches.add(id);
}
```

Результат

```
{ items: [...],
  total: 57,
  limit: 20,
  offset: 40 }
```

total рахується в тому самому проході: окремий COUNT(*) не потрібен.

Сортування й пагінація

SQL

```
SELECT * FROM kids
WHERE age >= 3
ORDER BY last_name, age DESC
LIMIT 20 OFFSET 40;

SELECT count(*) FROM kids
WHERE age >= 3;
```

У НАС

```
await db.list('kids', {
  where: { age: { gte: 3 } },
  sort: 'lastName,-age',
  limit: 20, offset: 40,
});
// GET /api/kids?where={"age":{"gte":3}}
//   &sort=lastName,-age&limit=20&offset=40
```

Українська абетка

Intl.Collator('uk', { numeric: true }): «Група 2» стоїть перед «Група 10»

null — у кінці

значення null і відсутні поля завжди в кінці, в обох напрямках

Стабільність

рівні значення сортуються за id, тож пагінація детермінована

Діапазон = сортування

gte/lt по рядку дає суцільний відрізок відсортованого списку

JOIN-ів немає: дані збираються окремими запитами

SQL

```
SELECT g.*, k.*
FROM groups g
LEFT JOIN kids k ON k.group_id = g.id
WHERE g.id = $1;

-- один запит, один знімок даних
```

У НАС

```
// GroupRepository
async findWithKids(id) {
  return this.uow.run(async () => {  // один знімок
    const group = await this.findById(id);
    if (!group) return null;
    const { items: kids } = await this.db.list(
      'kids', { where: { groupId: id } });
    return { ...group, kids };
  });
}
```

Без uow.run

Між findById і list інший запит може перемістити дитину, і результат зібрано з двох різних станів бази.

Без N+1

Батьків кількох дітей беремо одним запитом: inList('id', ids). Він теж іде через індекс.

Вкладені документи (MongoDB) чи нормалізація

Порада: вкладати, як у MongoDB

```
// groups/….json
{ "name": "Сонечко",
  "kids": [
    { "firstName": "Оля",
      "mother": { "firstName": "Ірина",
                  "phone": "+380…" } },
    { "firstName": "Петро",   // брат Олі
      "mother": { "firstName": "Ірина",
                  "phone": "+380…" } } ] }
```

- Ірина записана двічі: змінили телефон в одному місці, а в іншому лишився старий
- Перевести дитину в іншу групу = переносити піддокумент між файлами
- Нотатки, кімнати, вчителі посилаються на дитину, але дитина не має власного файлу
- Документ групи росте: повертаємося до проблем «таблиця = файл»

Обрано: нормалізація (ЗНФ)

```
// kids/0190…a1.json
{ "firstName": "Оля",
  "groupId": "0190…g1", "motherId": "0190…p1" }

// kids/0190…b2.json
{ "firstName": "Петро",
  "groupId": "0190…g1", "motherId": "0190…p1" }

// parents/0190…p1.json — один раз
{ "firstName": "Ірина", "phone": "+380…" }
```

- Кожен факт зберігається один раз: немає аномалій оновлення
- Зв’язки — це id, і їхню цілісність перевіряє рушій
- Платимо ручними JOIN-ами: окремими запитами в транзакції

Менеджер зв’язків: зовнішні ключі без SQL

SQL

```
CREATE TABLE kids (
  group_id UUID REFERENCES groups (id)
    ON DELETE SET NULL
);
CREATE TABLE notes (
  kid_id UUID NOT NULL
    REFERENCES kids (id)
    ON DELETE RESTRICT
);
```

У НАС

```
// kids/config.json
"groupId": { "type": "string", "nullable": true,
  "references": { "collection": "groups",
                  "onDelete": "setNull" } }
// notes/config.json
"kidId": { "type": "string", "required": true,
  "references": { "collection": "kids" } }
  // onDelete за замовчуванням: restrict
```

onDelete	Для	Що робить	SQL
`restrict (за замовч.)`	рядок, масив	409 REFERENCE_RESTRICT і список тих, хто посилається	`ON DELETE RESTRICT`
`setNull`	рядок (nullable)	посилання стає null	`ON DELETE SET NULL`
`remove`	масив посилань	id прибирається з масиву	`DELETE рядка зі зв’язувальної таблиці`

Що гарантує менеджер зв’язків

1. **`create / update`** — посилання — null або id існуючого запису, інакше 422 REFERENCE_NOT_FOUND
2. **`delete`** — onDelete застосовується в тій самій транзакції; повертається звіт, кого зачепило
3. **`старт бази`** — биті посилання (наприклад, файл видалили вручну) → BROKEN_REFERENCES, сервер не стартує
4. **`1 : 1`** — unique + references: одна група — один вихователь, одна кімната — одна група

ЗВІТ DELETE

```
const report = await db.delete('groups', groupId);
// {
//   deleted: [{ collection: 'groups', id: groupId }],
//   updated: [
//     { collection: 'kids', id: '0190…a1',
//       fields: ['groupId'] },
//     { collection: 'rooms', id: '0190…r3',
//       fields: ['groupId'] }
//   ]
// }
```

Інший бік зв’язку не синхронізується

Зв’язок зберігається з одного боку. Зворотний бік знаходимо запитом: contains('childIds', kidId) чи eq('motherId', id).

Транзакції: BEGIN / COMMIT / ROLLBACK

SQL

```sql
BEGIN;

INSERT INTO groups (name, age)
VALUES ('Сонечко', 3)
RETURNING id;

INSERT INTO kids (first_name, group_id)
VALUES ('Оля', $1);

COMMIT;   -- або ROLLBACK;
```

У НАС

```js
const group = await db.transaction(async (tx) => {
  const group = await tx.create('groups',
    { name: 'Сонечко', age: 3 });
  await tx.create('kids',
    { ...kid, groupId: group.id });
  return group;              // → COMMIT
  // throw error             // → ROLLBACK, помилка летить далі
  // return tx.rollback(x) // → ROLLBACK, повертає x
});
// одна дитина не пройшла перевірку → немає ні групи, ні дітей
```

return коміт: усі зміни записуються атомарно

throw відкат: жодна зміна не записується

tx.rollback(v) відкат зі значенням: «пробний запуск»

Як транзакція влаштована всередині

Буфер транзакції (CollectionView)

Map<id, запис | null>: створені, змінені, видалені (null)

читання: спершу буфер, потім база

Закомічена колекція (Collection)

id та індекси в RAM, записи — у файлах. Звичайні запити бачать лише цей шар.

- **Свої зміни видно.** Щойно створений запис знаходиться через tx.get / tx.list, і на нього можна послатися.
- **Предикати ті самі.** CollectionView має той самий інтерфейс, що й Collection, тож QueryContext і RelationManager працюють і з незакоміченими даними.

Правила

- **Тайм-аут:** 10 с за замовчуванням, { timeout } для окремої транзакції; після нього відкат і 503 TRANSACTION_TIMEOUT.
- **Лише tx:** виклик db.* усередині транзакції → INVALID_TRANSACTION_USE (захист від взаємоблокування).
- **tx живе, поки йде транзакція:** після неї → TRANSACTION_CLOSED.
- **По черзі:** операції однієї транзакції виконуються послідовно навіть у Promise.all; забуті await дочікуються перед комітом.
- **Перехоплена помилка операції** відкочує лише цю операцію, а не всю транзакцію.

Приклад: не більше 20 дітей у групі (BR-03)

SQL

```
BEGIN ISOLATION LEVEL SERIALIZABLE;

SELECT count(*) FROM kids
WHERE group_id = $1;
-- якщо count >= 20 → ROLLBACK

UPDATE kids SET group_id = $1
WHERE id = $2;

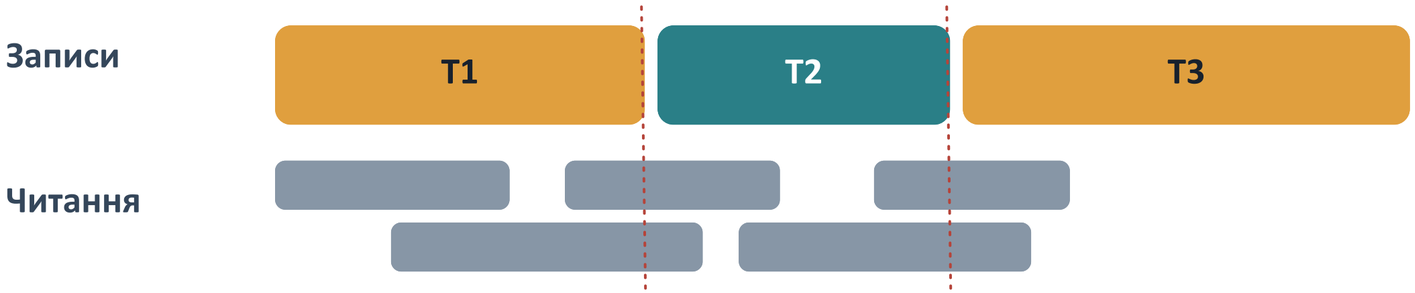
COMMIT;
-- може впасти з 40001
-- serialization_failure → повторити
```

У НАС

```
await uow.run(async () => {
  const total = await kids.countByGroup(groupId);
  if (total >= MAX_KIDS_IN_GROUP) {
    throw new HttpError(StatusCodes.CONFLICT,
      'GROUP_IS_FULL', 'The group is full.');
  }
  await kids.update(kidId, { groupId });
});
// 21-ша дитина → 409, база не змінилась
// повторювати не треба: конфліктів не буває
```

NFR-10: недійсні операції (21-ша дитина, накладання занять) не повинні змінювати базу. Транзакція гарантує це автоматично.

Рівень ізоляції — завжди SERIALIZABLE



пунктир — момент застосування коміту: лише тут читання на мить чекають

Чому так

- Простий код: без MVCC, взаємоблокувань і повторів транзакцій
- Домен садка: сотні записів, мало одночасних записувачів, короткі транзакції
- Бізнес-правила (ліміт 20, розклад без накладань) гарантуються без додаткових блокувань

SQL

```
SET TRANSACTION ISOLATION LEVEL
  SERIALIZABLE;
-- PostgreSQL за замовчуванням:
-- READ COMMITTED
-- конфлікт → 40001, клієнт повторює
```

У НАС

```
// readers-writer lock (lock-manager.js)
// черга FIFO: записувач не «голодує»
// одна транзакція-записувач за раз
// конфліктів немає → повторів немає
```

Ціна: паралельних записів немає, тож повільна транзакція тримає чергу. Тому в транзакції не роблять нічого повільного (зовнішніх викликів), а тайм-аут обриває завислі.

Unit of Work: вкладені транзакції

```
// KidService.deleteKid (FR-018, спрощено)
async deleteKid(kidId) {
  return this.#uow.run(async () => {     // відкриває
    const kid = await this.#kids.getById(kidId);
    await this.#notes.deleteByKid(kidId); // приєднується
    await this.#kids.delete(kidId);
    for (const id of [kid.motherId, kid.fatherId]
                       .filter(Boolean)) {
      // бачить видалення вище
      if (await this.#kids.countByParent(id) === 0)
        await this.#parents.delete(id);
    }
    return kid;
  });                                    // COMMIT тут
}
```

KidService.deleteKid → run (відкриває)

NoteRepository.deleteByKid → run (приєднується)

KidRepository.countByParent → run (приєднується)

parents.delete, kids.delete → executor() = tx

одна транзакція, один коміт

- **Спільний інтерфейс:** uow.run для сервісів і репозиторіїв, uow.executor() повертає tx усередині run і db поза ним.
- **Перша помилка перемагає:** після збою вкладеного run транзакція «приречена», і назовні виходить саме ця помилка.

Атомарний запис файлу

1 writeFile

`<id>.json.<uuid>.tmp`

дані йдуть у тимчасовий файл

2 fsync(file)

`вміст на диску`

дані не лишаються в кеші ОС

3 rename

`tmp → <id>.json`

атомарна заміна на рівні ОС

4 fsync(dir)

`запис у директорії`

без цього rename може зникнути після вимкнення живлення

SQL

```
-- PostgreSQL робить це сам:
-- сторінки даних, fsync, WAL.
-- Розробник цього не бачить.
```

У НАС

```
// src/storage/fs-utils.js — atomicWriteJson
await handle.writeFile(json);
await handle.sync();
await fs.rename(temporary, target);
await syncDirectory(path.dirname(target));
```

Журнал транзакцій (write-ahead log)

1. **`journal.json`**
   усі зміни транзакції записуються атомарно: це точка коміту
2. **`файли записів`**
   кожна зміна застосовується: запис цілком або видалення
3. **`видалити журнал`**
   коміт завершено

Процес упав…	Що буде під час старту
до кроку 1	транзакції не було; .tmp прибирається
між 1 і 3	журнал застосовується повторно
диск зламався на кроці 2	503 DATABASE_NEEDS_RECOVERY до перезапуску

```
// database/journal.json
{
  "version": 1,
  "changes": [
    { "collection": "kids",
      "id": "0190…a1",
      "record": { "firstName": "Оля",
                  "groupId": null, … } },
    { "collection": "groups",
      "id": "0190…g1",
      "record": null }   // видалення
  ]
}
```

Ідемпотентність: у журналі повні записи, а не різниця, тож повторне застосування дає той самий результат.

Одна директорія — один сервер

```
// database/.lock
{
  "pid": 48213,
  "threadId": 0,
  "hostname": "kindergarten-api",
  "startedAt": "2026-11-03T09:41:12.000Z",
  "token": "6f1c…"
}
```

Сервер викликає close() на SIGTERM і SIGINT (Ctrl+C) після зупинки HTTP-сервера.

Другий сервер

на тій самій директорії не стартує: 503 DATABASE_LOCKED

Другий FileDatabase

у тому самому процесі — теж DATABASE_LOCKED

Аварія

lock процесу, що впав, виявляється за PID і перехоплюється

close()

нові виклики → 503 DATABASE_CLOSED, поточні доходять до кінця, потім lock звільняється

Пайплайн ініціалізації: init()

1 Зайняти директорію
.lock

→

2 Прочитати конфіги
config.json → INVALID_CONFIG

→

3 Граф зв’язків
RelationManager

→

4 Завантажити id та індекси
з файлів записів

→

5 Застосувати журнал
якщо процес упав посеред коміту

→

6 Перевірити посилання
BROKEN_REFERENCES

→

7 Знайти дублікати unique
попередження

Попередження	Коли
INVALID_RECORD	запис не відповідає поточній схемі (наприклад, після зміни config.json)
DUPLICATE_VALUE	значення в unique-колонці повторюється (unique додали до колонки з даними)

```
const db = new FileDatabase();
await db.init();
for (const w of db.warnings) {
  // { code: 'INVALID_RECORD',
  //   collection: 'kids', id: '…' }
}
```

Помилки й логування

`DatabaseError { code, status, message }`

4xx — **проблема в запиті**

```
400 INVALID_QUERY · 404 NOT_FOUND
409 UNIQUE_VIOLATION · 422 VALIDATION_ERROR
```

500 — **конфігурація, дані чи код**

```
INVALID_CONFIG · CORRUPT_JOURNAL
INVALID_TRANSACTION_USE
```

503 — **тимчасовий стан бази**

```
DATABASE_LOCKED · TRANSACTION_TIMEOUT
DATABASE_NEEDS_RECOVERY
```

SQL

```
-- SQLSTATE 23505
-- unique_violation
```

У НАС

```
409 UNIQUE_VIOLATION
```

Логування

- **Логер передається ззовні:** pino, дочірній логер або навіть console.
- **Вміст записів не логується ніколи:** лише колекції, id, кількість змін, тривалість і коди. У базі особисті дані дітей.
- **requestId** додається до всіх рядків логу запиту, зокрема й до рядків рушія.

```
{"level":20,"module":"storage",
 "requestId":"5c2e…","changes":1,
 "collections":{"kids":1},"ms":3,
 "msg":"Transaction committed"}
```

НЕ РЕАЛІЗОВАНО — ІДЕЯ

Новий челендж: журнал усіх змін

commit()

журнал → файли записів

→

logs/{uuidv7}.json

один файл на коміт

→

Збирач

раз на N хвилин

→

audit-2026-11.jsonl

rolling: за днем / розміром

```
// logs/0193a1c4-7e2f-7b10-….json
{
  "committedAt": "2026-11-03T09:41:12.000Z",
  "requestId": "5c2e…",
  "user": "…",
  "changes": [
    { "collection": "kids", "id": "0190…a1",
      "op": "update",
      "before": { "groupId": "0190…g1" },
      "after":  { "groupId": null } }
  ]
}
```

Чому це нескладно

- **Одна точка:** усі зміни, і наші, і від onDelete, вже зібрані в changes() на коміті.
- **Порядок безкоштовно:** UUIDv7 у назві, тож ls дає хронологію.
- **Без конфліктів:** кожен коміт пише новий файл, нічого не дописується.
- **Збирач:** зливає файли в один, видаляє злиті, ротує за днем чи розміром і тримає N останніх.
- **Застереження:** тут будуть особисті дані, тож logs/ захищаємо так само, як database/.

Свідомі обмеження

Немає: JOIN

пов’язані записи читаються окремими запитами в транзакції

Немає: Каскадне видалення

наслідки понад setNull / remove реалізуються в сервісах

Немає: Міграції та DDL

поступове виправлення записів через PATCH

Немає: Паралельні записи

один записувач, SERIALIZABLE; читання паралельні

Немає: Складений unique

і unique без урахування регістру: нормалізуємо в сервісі

Немає: Записи в RAM

фільтр без індексу й сортування читають кандидатів з диска

Підсумок

Структура

колекція = директорія, запис = файл, схема — config.json

Запити

предикати, індекси в RAM, сортування й пагінація

Цілісність

зв’язки з onDelete, unique, перевірка під час старту

Транзакції

SERIALIZABLE, Unit of Work для вкладених

Надійність

атомарні файли, журнал, .lock, відновлення